

Caderno de Arquitetura

1. Propósito:

O Caderno de Arquitetura define a filosofia, as decisões, as restrições, as justificações e os elementos significativos que orientam o design e a implementação do sistema. O objetivo deste documento é estabelecer uma visão arquitetônica clara e consistente, que sirva de referência para toda a equipa ao longo do desenvolvimento da solução.

A arquitetura aqui descrita define os princípios fundamentais que guiam a construção do sistema, identifica os aspetos críticos que influenciam as decisões técnicas e estabelece os limites e diretrizes que asseguram coerência, qualidade e sustentabilidade. Este documento também clarifica como os componentes se relacionam, como serão integrados e quais os mecanismos essenciais que suportam o funcionamento do sistema.

O propósito deste caderno é, portanto, garantir que todas as decisões arquitetónicas são compreendidas, justificadas e alinhadas com os requisitos funcionais e não funcionais do projeto, permitindo que a equipa trabalhe com uma base comum e que o sistema evolua de forma estruturada e controlada.

Elemento	Descrição
Objetivo do Documento	Servir como documento técnico orientador para a equipa responsável pelo desenvolvimento do sistema
Destinatários	Programadores, equipa técnica do projeto, docentes da unidade curricular e responsável pela gestão do projeto
Frequência de Revisão	Atualização a quinze dias ou sempre que ocorram alterações arquiteturais relevantes
Responsáveis pela Manutenção	Arquiteto do sistema em colaboração com os analistas

2. Objetivos Filosóficos e Arquitetónicos:

A arquitetura do sistema é orientada por uma filosofia que privilegia a simplicidade, a organização e a facilidade de evolução. O objetivo é garantir que o sistema de bilhética funciona

de forma clara, segura e eficiente, permitindo gerir operações como compras, carregamentos, validações e consulta de dados sem criar complexidade desnecessária.

A solução deve ser construída de forma modular, com cada parte do sistema a ter responsabilidades bem definidas. Isto facilita o trabalho da equipa, reduz erros e torna o sistema mais fácil de manter ao longo do tempo. Como o projeto envolve dados sensíveis e operações que exigem consistência, a arquitetura deve assegurar que estas ações são tratadas de forma fiável e controlada.

Os principais objetivos arquitetónicos são:

Organização clara do sistema — Separar bem o frontend, backend, APIs e base de dados, garantindo que cada camada tem o seu papel e que o código é fácil de compreender.

Facilidade de manutenção — Usar padrões simples e boas práticas que permitam corrigir problemas e adicionar novas funcionalidades sem reescrever grandes partes do sistema.

Crescimento natural — Permitir que o sistema suporte mais utilizadores, mais transações e mais dados no futuro, sem comprometer o desempenho.

Integração simples entre componentes — Garantir que o frontend e o backend comunicam de forma consistente através de APIs REST, facilitando também integrações futuras com outros sistemas dos TUB.

Fiabilidade nas operações de bilhética — Assegurar que compras, carregamentos e validações são registados corretamente e que os dados permanecem coerentes.

Segurança básica dos dados — Proteger informações de utilizadores e transações através de autenticação, validação de dados e boas práticas de acesso.

Simplicidade tecnológica — Utilizar tecnologias familiares à equipa (Java, React, MySQL) para reduzir complexidade e acelerar o desenvolvimento.

Transparência e previsibilidade — Garantir que o comportamento do sistema é fácil de entender e que as decisões arquitetónicas são justificadas e documentadas.

Estes objetivos servem como guia para todas as decisões técnicas tomadas ao longo do projeto e ajudam a garantir que a solução final é sólida, compreensível e adequada às necessidades do sistema de bilhética.

3. Suposições e Dependências

As suposições e dependências desempenham um papel fundamental na definição da arquitetura do projeto, pois influenciam diretamente as decisões tecnológicas, a viabilidade da implementação do sistema e os desafios esperados ao longo do desenvolvimento. Neste ponto,

são listadas as principais suposições feitas sobre o ambiente, os requisitos e recursos disponíveis, bem como as dependências que podem afetar a arquitetura. A definição clara destes elementos permite antecipar riscos, alinhar expectativas e garantir que a evolução do projeto está sustentada sobre uma base sólida e bem planeada.

Suposições:

- O TickeTUB expõe um endpoint REST sobre HTTPS para o SCB submeter picagens em formato JSON, com os campos obrigatórios: timestamp (ISO 8601), ID da linha, tipo de título, local de validação (GPS ou paragem) e ID do equipamento; as credenciais de autenticação do SCB estão pré-configuradas;
- O Auth0 está configurado e acessível com domínio, client ID e redirect URI definidos; o backend Spring Boot está configurado com o issuer-uri correto para validação de tokens JWT;
- Os alertas operacionais (validações inválidas, cartões expirados, duplicações) serão detetáveis em tempo real com latência de notificação inferior a 30 segundos;
- Existe conectividade de rede estável nos veículos e paragens, assegurando a ingestão contínua de dados para a plataforma;
- Os dados de bilhética poderão ser anonimizados e pseudonimizados em conformidade com o RGPD, sem comprometer a utilidade analítica da informação;
- Os modelos de dados serão compatíveis com os Smart Data Models da iniciativa FIWARE e exportáveis em formato NGSI-LD;
- O SAE-IP está operacional e a fornecer coordenadas GPS dos veículos com tolerância de 2 segundos face ao timestamp de validação;

Dependências:

- O sistema depende do SCB como fonte primária de dados de validação — o SCB envia picagens via HTTP POST para o endpoint de ingestão do TickeTUB; sem essa conexão estável o pipeline (UC02) não funciona;
- Auth0 (IDP externo) é uma dependência crítica para autenticação SSO (UC01); a sua indisponibilidade impede qualquer acesso ao sistema;
- Dependência do SAE-IP para georreferenciação de validações (UC07) e enriquecimento da matriz O-D (UC08);
- Feeds GTFS e GTFS-RT para correlacionamento entre procura real e oferta planeada (UC07);

- OpenStreetMap via Leaflet para visualização cartográfica no Mapa de Rede (UC06);
- ERP dos TUB para integração financeira e controlo de receitas (UC11);
- Conformidade com o RGPD como dependência legal, condicionando anonimização, retenção e partilha de dados;
- GitHub para controlo de versão e colaboração entre os membros da equipa.

4. Requisitos arquiteturais relevantes

Os requisitos arquitetonicamente significativos influenciam diretamente as decisões de design do sistema, condicionando a sua estrutura, desempenho, interoperabilidade e segurança. Estes requisitos derivam diretamente dos casos de uso UC01–UC12.

Requisito	Descrição
Autenticação SSO (UC01)	Toda a autenticação é delegada ao Auth0 via OAuth2/OIDC. O backend Spring Boot valida tokens JWT e atribui perfis RBAC. Sem log de auditoria, sem acesso.
Ingestão em tempo real (UC02)	O TickeTUB expõe um endpoint REST (HTTPS) para o SCB submeter picagens via HTTP POST. O sistema autentica o SCB, valida a estrutura JSON, responde com HTTP 202 (assíncrono) e processa os dados: validação $\leq 5s$, normalização $\leq 10s$ e persistência $\leq 15s$. Rate limiting e suporte a backoff obrigatórios.
Anonimização RGPD (UC02, UC03)	Pseudonimização unidirecional obrigatória antes de qualquer armazenamento. Nenhum identificador pessoal real pode persistir nos dados normalizados.
Categorização por perfil (UC03)	Classificação automática por tipologia de título (estudante, sénior, mensal, avulso, gratuidade). Tabela de mapeamento editável sem intervenção de desenvolvimento.
Dashboard operacional (UC04)	Dashboard operacional (UC04) com widgets de procura em tempo real, estado de ingestão e alertas activos. Conteúdo adaptado ao

	perfil RBAC. Carregamento dos widgets críticos dentro do SLA definido.
Procura em tempo real (UC05)	Análise integrada da procura em tempo real nas perspetivas de horário, rota/linha e zona/paragem. Atualização contínua com filtros combinados. Tempo de resposta dentro do SLA operacional definido.
Mapa interativo (UC06)	Mapa OpenStreetMap via Leaflet com marcadores georreferenciados de todas as paragens TUB e popups LIVE DATA com dados em tempo real. Carregamento inicial em ≤ 5 segundos. Degradação graciosa em indisponibilidade OSM.
Correlação GPS/SAE-IP (UC07)	Cruzamento de validações com posicionamento GPS em tempo real (tolerância 2s). Cobertura de eventos sem localização exata não deve exceder 5% diários.
Matriz O-D (UC08)	Cálculo diário da matriz Origem-Destino disponível antes das 07h00, com agregação por período e limiar mínimo de privacidade nos pares O-D exportados.
Alertas em tempo real (UC09)	Deteção automática de anomalias (título expirado, duplicação, evasão) com notificação em menos de 30 segundos. Regras configuráveis sem intervenção de desenvolvimento.
Histórico e KPIs (UC10)	Consolidação diária automática no Data Lake com suporte a pelo menos 7 anos de histórico. Consultas comparativas em menos de 10 segundos para granularidade mensal.
Integração ERP (UC11)	Transferência automática de dados financeiros consolidados para o ERP com retry automático a cada 15 minutos em caso de falha.
Exportação NGSI-LD (UC12)	API RESTful documentada em OpenAPI 3.0, compatível com NGSI-LD. Exportação em CSV/JSON bloqueada se dados não anonimizados forem detetados.

5. Decisões, restrições e justificações

As seguintes decisões e restrições arquiteturais foram definidas para orientar o desenvolvimento do TickeTUB, garantindo coerência técnica e alinhamento com os requisitos dos casos de uso.

Decisão	Justificação
Java + Spring Boot (backend)	Obrigatório no contexto do projeto. Oferece ecossistema robusto para APIs RESTful, validação de tokens JWT, integração com bases de dados e suporte a pipeline ETL.
React (frontend)	Permite construção de interfaces dinâmicas e reativas, adequadas para dashboards em tempo real, módulo de procura em tempo real, mapa interativo Leaflet e popups LIVE DATA (UC04, UC05, UC06).
MySQL (base de dados)	Garante consistência e integridade dos dados operacionais, perfis RBAC, logs de auditoria e configurações do sistema.
Auth0 como IDP externo	A autenticação é totalmente delegada ao Auth0, evitando que a aplicação gire passwords diretamente. Suporta OAuth2/OIDC, SSO e RBAC por roles (UC01).
Data Lake para persistência analítica	Repositório NGSI-LD para dados normalizados e histórico de validações. Suporta particionamento temporal, índices geoespaciais e políticas de backup (UC02, UC08, UC10).
Arquitetura monolítica	Dado o âmbito e escala do projeto, uma arquitetura monolítica é suficiente e reduz a complexidade operacional, facilitando desenvolvimento, testes e execução local.
OpenStreetMap via Leaflet	Solução open-source para o mapa interativo (UC06), com degradação graciosa em caso de indisponibilidade dos tiles OSM.

Desenvolvimento manual em Linux	A equipa construiu toda a solução manualmente, sem frameworks de geração automática. Todo o ambiente foi configurado em Linux com instalação manual de Java, Node.js, MySQL e ferramentas de build.
Anonimização unidirecional	A pseudonimização é aplicada no pipeline ETL (UC02.2) com chave gerida pelo DPO, não reversível via API. É tecnicamente impossível exportar dados não anonimizados (UC12).
Fail secure em RBAC	Um utilizador sem perfil atribuído nunca obtém acesso parcial. Sem log de auditoria persistido, o acesso é bloqueado. Princípio aplicado em UC01 e UC04.

6. Mecanismos arquitetónicos

Os mecanismos arquitetónicos são soluções estruturais que suportam o funcionamento do sistema. Cada mecanismo corresponde a uma ou mais capacidades implementadas nos casos de uso UC01–UC12.

6.1 Mecanismo de Autenticação e Autorização (UC01)

Responsável pela autenticação via SSO com Auth0 como IDP externo. Inclui redirecionamento HTTPS para o Auth0, validação de tokens JWT (assinatura, expiração, issuer e audience), atribuição de perfis RBAC por roles extraídos do token, e registo automático de logs de auditoria para todos os eventos de autenticação. A validação do token deve completar em menos de 1 segundo. Princípio fail secure: sem perfil válido, sem acesso; sem log persistido, sem acesso.

6.2 Mecanismo de Ingestão via API REST (UC02)

O TickeTUB expõe um endpoint REST sobre HTTPS para o SCB submeter picagens via HTTP POST. O sistema autentica e autoriza o SCB, valida a estrutura do payload JSON e responde imediatamente com HTTP 202 (Accepted), indicando que os dados foram aceites para processamento assíncrono. O processamento decorre em quatro fases:

- **Fase 1 — Validação (UC02.1):** verificação estrutural (tipos de dados, timestamp ISO 8601, linha_id no catálogo, tipo_titulo em lista aprovada), deteção de duplicados por cache dos últimos 5 minutos, e validação de regras de negócio. Registos inválidos vão para quarentena com motivo detalhado sem bloquear os válidos. Cobertura de 100% dos registos em ≤ 5 segundos.
- **Fase 2 — Normalização (UC02.2):** pseudonimização RGPD unidirecional, mapeamento SCB $\rightarrow$ SmartDataModels, serialização NGSI-LD com metadados de versão ETL. Nenhum identificador pessoal persiste. Tempo ≤ 10 segundos.
- **Fase 3 — Armazenamento (UC02.3):** persistência assíncrona no Data Lake com particionamento diário, atualização de índices geoespacial e temporal, verificação de integridade pós-escrita. Fila persistente com retry automático (5 tentativas / 30s) em caso de indisponibilidade do Data Lake. Tempo ≤ 15 segundos. Se a indisponibilidade persistir, o estado passa a “Falha Técnica” e o Administrador Técnico é notificado.

6.3 Mecanismo de Categorização por Perfil (UC03)

Classificação automática de cada evento de validação por tipologia de título (estudante, sénior/reformado, passe mensal, avulso/ocasional, gratuidade, não categorizado). A tabela de mapeamento é editável pelo administrador de TI sem intervenção de desenvolvimento. O DPO define e audita as políticas de anonimização e aprova exportações de dados para fins académicos ou institucionais. Eventos não categorizados vão para fila de revisão semanal.

6.4 Mecanismo de Dashboard Operacional (UC04)

Após autenticação bem-sucedida, o utilizador acede à dashboard principal que apresenta os blocos operacionais da plataforma: métricas resumidas de procura por janela temporal e linhas críticas, estado da ingestão em tempo real (normal / degradado / falha), alertas ativos com severidade e categoria, e atalhos para os módulos operacionais e analíticos. O conteúdo é adaptado ao perfil RBAC do utilizador. O logout invalida a sessão no servidor e revoga o token. Em caso de expiração de sessão, o utilizador é redirecionado para autenticação. Falhas parciais de widgets são tratadas de forma isolada sem bloquear o restante ecrã.

6.5 Mecanismo de Procura em Tempo Real (UC05)

Análise integrada da procura em tempo real nas três perspetivas principais: por horário (identificação de picos e hora de maior afluência), por rota/linha (comparação com média histórica e cálculo de desvio percentual) e por zona/paragem (agregação georreferenciada com atualização contínua). Suporta filtros combinados temporais e espaciais. Dados indisponíveis numa perspetiva apresentam última atualização válida com timestamp. Paragens sem coordenadas são sinalizadas sem impedir a análise nas restantes perspetivas.

6.6 Mecanismo de Mapa de Rede (UC06)

Mapa interativo baseado em OpenStreetMap renderizado via biblioteca Leaflet, centrado em Braga. Apresenta marcadores georreferenciados de todas as paragens TUB. Ao clicar num marcador, um popup LIVE DATA exhibe: nome da paragem, hora de maior afluência, taxa de afluência, validações inválidas (número e percentagem), distribuição por tipo de título e total de validações. Em caso de indisponibilidade dos tiles OSM, os marcadores e dados de bilética mantêm-se acessíveis (degradação graciosa). Carregamento inicial em ≤ 5 segundos.

6.7 Mecanismo de Correlação Operacional (UC07)

Cruzamento em tempo real de eventos de validação com posicionamento GPS fornecido pelo SAE-IP (tolerância de 2 segundos). Cada validação recebe um indicador de qualidade geográfica: 'localização exata', 'localização estimada' ou 'correlação por horário planeado'. Análise periódica de desvios entre serviço planeado (GTFS) e executado, com destaque automático de linhas com desvio acima do limiar configurável. Alertas quando cobertura de localização exata cai abaixo de 95%.

6.8 Mecanismo de Matriz Origem-Destino (UC08)

Cálculo diário automático da matriz O-D a partir de sequências de validações georreferenciadas. Destino estimado por prioridade: (a) validação seguinte do mesmo título pseudonimizado, (b) terminus da linha como fallback. Matriz disponível antes das 07h00, agregada por período (ponta, vazio, fim de semana) com indicador de confiança. Pares O-D exportados sempre acima de limiar mínimo de privacidade. Visualização no mapa com código de cores por ocupação.

6.9 Mecanismo de Alertas e Exceções (UC09)

Monitorização contínua de eventos de validação com três regras de deteção: título expirado, duplicação no mesmo veículo/período e título suspenso/inválido. Alertas gerados em menos de 30 segundos com severidade e timestamp. Volume crítico de alertas (>50/hora numa linha) escalado automaticamente para o Centro de Controlo. Regras e limiares configuráveis sem intervenção de desenvolvimento. Registo auditável de todas as ações de gestão de alertas.

6.10 Mecanismo de Histórico e Consolidação (UC10)

Consolidação diária automática no horário agendado (ex.: 23h30), agregando eventos por linha, veículo, perfil tarifário, hora, dia da semana e canal de venda. Cálculo de KPIs diários: total de validações, receita estimada por linha e título, taxa de anomalias e ocupação média por viagem. Data Lake com suporte mínimo de 7 anos de histórico. Consultas comparativas com destaque automático de desvios superiores a $\pm 10\%$. Reprocessamento parcial suportado sem recalcular a totalidade dos dados.

6.11 Mecanismo de Integração com Planeamento e ERP (UC11)

Módulo de simulação de cenários de ajuste de rede com projeção de impacto e indicador de nível de confiança. Todos os cenários ficam registados com identificação do utilizador, data e versão dos dados. Transferência automática diária de dados financeiros consolidados para o ERP (receita por linha e título, validações por canal, anomalias com impacto financeiro) com retry automático a cada 15 minutos em caso de falha. Comparação automática entre receita realizada e valores contratualizados com destaque de desvios.

6.12 Mecanismo de Exportação e Interoperabilidade (UC12)

API RESTful documentada em OpenAPI 3.0, compatível com NGSI-LD e Smart Data Models. Autenticação via OAuth 2.0 sobre HTTPS. Exportação de dados em CSV e JSON restrita a conjuntos aprovados pelo DPO. Verificação técnica obrigatória de anonimização antes de qualquer exportação — é tecnicamente impossível exportar dados não anonimizados independentemente do perfil. Log de exportações imutável e acessível ao DPO.

7. Abstrações-chave

As abstrações representam os conceitos fundamentais do sistema TickeTUB e são a base para o modelo de domínio, os casos de uso e a própria arquitetura.

Abstração	Descrição
Utilizador / Funcionário TUB	Entidade que interage com a plataforma. Perfis: Gestor de Operações, Analista de Dados, DPO, Administrador de TI, Equipa de Fiscalização, Gestor de Planeamento, Direção. Credenciais geridas pelo Auth0; permissões por RBAC.
Evento de Validação (Picagem)	Registo de utilização de um título de transporte. Atributos obrigatórios: timestamp, ID da linha, tipo de título, localização (GPS/paragem) e ID do equipamento. Unidade fundamental do pipeline de ingestão.
Título de Transporte	Representa o direito de viagem. Tipos: passe estudante, sénior/reformado, mensal, avulso/ocasional, gratuidade. Base para categorização por perfil (UC03) e deteção de anomalias (UC09).
Perfil Tarifário	Classificação do utilizador baseada na tipologia do título. Permite análise segmentada da procura sem identificação de passageiros (RGPD). Gerido pela tabela de mapeamento editável pelo administrador.
Entidade NGSI-LD	Representação normalizada de um evento de validação após transformação ETL, compatível com Smart Data Models (Trip, TransitStop). Unidade de armazenamento no Data Lake.
Matriz Origem-Destino	Agregação de pares O-D estimados a partir de sequências de validações georreferenciadas. Principal entregável estratégico do sistema para planeamento da rede.
Alerta / Anomalia	Registo de desvio detetado no processo de validação (expirado, duplicado, suspenso, evasão). Inclui severidade, timestamp, regra ativada e estado de tratamento. Gerido pela equipa de fiscalização.

Paragem TUB	Nó georreferenciado da rede de transportes com coordenadas GPS precisas. Base para o Mapa de Rede (UC06), georreferenciação de validações (UC07) e matriz O-D (UC08).
Log de Auditoria	Registo imutável de todos os eventos críticos do sistema (autenticação, ingestão, exportação, alertas). Conservado por mínimo de 12 meses. Acessível exclusivamente ao DPO e administradores.
Token JWT	Credencial de sessão emitida pelo Auth0 com claims de roles RBAC. Validado em cada pedido ao servidor. Nunca armazenado em logs nem exposto em respostas de API.
Data Lake	Repositório central de dados normalizados NGSI-LD com particionamento diário, índices geoespaciais e temporais e suporte a mínimo de 7 anos de histórico.
SCB (Sistema Central de Bilhética)	Fonte primária de dados. Envia picagens via HTTP POST para o endpoint REST do TickeTUB. Dependência crítica do pipeline de ingestão (UC02).

8. Camadas ou estrutura arquitetónica

A arquitetura do sistema segue um modelo em camadas, garantindo separação de responsabilidades e coerência com os casos de uso UC01–UC12.

Camada	Responsabilidade e Componentes
--------	--------------------------------

Apresentação (Frontend – React)	Dashboard operacional (UC04), módulo de procura em tempo real (UC05), Mapa de Rede com Leaflet/OSM e popups LIVE DATA (UC06), interface de autenticação (Auth0 SDK). Comunica exclusivamente via REST com a camada de API.
API / Controladores (Spring Boot)	Controladores REST para: autenticação/RBAC (UC01), ingestão ETL (UC02), categorização (UC03), dashboard (UC04), procura em tempo real (UC05), mapa (UC06), correlação operacional (UC07), O-D (UC08), alertas (UC09), histórico (UC10), planeamento/ERP (UC11), exportação NGSI-LD (UC12). Stateless. Validação JWT em cada pedido.
Domínio (Lógica de Negócio)	Motor de validação e normalização NGSI-LD (UC02), motor de categorização por perfil tarifário (UC03), motor de agregação de procura em tempo real (UC05), motor de correlação GPS/SAE-IP (UC07), algoritmo de estimativa O-D (UC08), motor de deteção de anomalias e alertas (UC09), motor de consolidação e KPIs (UC10), módulo de simulação de cenários (UC11). Serviço de anonimização RGPD transversal.
Infraestrutura (Conectores)	Conector SCB via API REST (UC02), Data Lake NGSI-LD com índices geoespaciais e temporais (UC02, UC08, UC10), Auth0 IDP externo (UC01), OpenStreetMap tiles via Leaflet (UC06), SAE-IP via API REST (UC07), GTFS/GTFS-RT feeds (UC07), ERP dos TUB via API (UC11), base de dados de logs de auditoria (UC01, UC02, UC09, UC12).
API Gateway	Ponto de entrada único. Autenticação OAuth2, encaminhamento de pedidos, logging e auditoria de acessos, transformação de requests/responses.
Processamento ETL	Endpoint REST (UC02): autenticação do SCB, HTTP 202 assíncrono, validação → normalização NGSI-LD → armazenamento Data Lake. Quarentena para dados inválidos, detecção de duplicados por cache (5 min), rate limiting, fila persistente com retry automático (5 tentativas / 30s), verificação de integridade pós-escrita.

Dados Operacionais (OLTP – MySQL)	Logs de autenticação e auditoria, configurações do sistema, tabela de mapeamento de perfis tarifários, catálogo de paragens, regras de alertas, metadados de ingestão.
Dados Analíticos (Data Lake / DW)	Entidades NGSI-LD particionadas por data, índices geoespaciais e temporais, séries temporais de KPIs, matrizes O-D históricas, logs de exportação imutáveis. Suporte mínimo de 7 anos.
Integração Externa	SCB (picagens), SAE-IP (GPS), Auth0 (IDP), OpenStreetMap (tiles), GTFS/GTFS-RT (horários), ERP (financeiro).

9. Vistas arquitetónicas

As vistas arquitetónicas descrevem o sistema a partir de diferentes perspetivas, permitindo compreender como está organizado, como funciona e como responde aos requisitos dos casos de uso UC01–UC12.

9.1 Vista Lógica

A Vista Lógica descreve a estrutura interna do sistema, a organização em camadas e as dependências críticas entre componentes. As dependências seguem sempre a direção descendente — nenhuma camada inferior invoca diretamente uma camada superior, garantindo baixo acoplamento e elevada testabilidade.

Interfaces e Dependências Críticas

- SCB → Endpoint de Ingestão TickeTUB: HTTP POST sobre HTTPS com payload JSON de picagens de bilhética (UC02)
- Auth0 → Backend Spring Boot: protocolo OAuth2/OIDC com token JWT assinado (UC01)

- Leaflet → OpenStreetMap: HTTP para tiles rasterizados com degradação graciosa (UC06)
- Motor de Normalização → Data Lake: escrita em formato NGSI-LD com Smart Data Models (UC02)
- Backend → SAE-IP: consulta de posicionamento GPS por veículo e timestamp (UC07)
- Backend → ERP: transferência automática de dados financeiros com retry (UC11)
-

9.2 Vista Operacional

A Vista Operacional descreve os nós físicos do sistema e como os componentes comunicam em produção.

Nó	Componentes e Comunicação
Browser (Cliente)	Frontend React, Auth0 SDK, Leaflet. HTTPS para API Layer; HTTP para tiles OSM.
Servidor de Aplicação	Spring Boot API, Domain Services, Pipeline ETL, Conectores. HTTPS REST com SCB e SAE-IP; OAuth2/OIDC com Auth0; REST com ERP.
Data Lake	Repositório NGSI-LD com partições diárias, índices geoespaciais e temporais. Acesso interno pelo servidor de aplicação.
MySQL (OLTP)	Logs de auditoria, configurações, catálogo de paragens, tabelas de mapeamento. Acesso interno pelo servidor de aplicação.
Auth0 (IDP Externo)	Serviço de autenticação e RBAC. OAuth2/OIDC com aplicação e browser.

SCB (Sistema Externo)	Sistema Central de Bilhética TUB. Envia picagens via HTTP POST para o endpoint de ingestão do TickeTUB sobre HTTPS.
SAE-IP (Sistema Externo)	Posicionamento GPS da frota. API REST sobre HTTPS para correlação operacional.
ERP (Sistema Externo)	Sistema de gestão financeira TUB. API REST para integração de dados consolidados.
OpenStreetMap (Externo)	Tiles de mapa base. HTTP via Leaflet a partir do browser.

Processos Principais em Execução Contínua

- Processo de Ingestão via API REST (UC02): endpoint HTTPS recebe HTTP POST do SCB, autentica, responde com HTTP 202 e processa de forma assíncrona: validação, normalização NGSI-LD e persistência no Data Lake.
- Processo de Correlação GPS (UC07): cruzamento em tempo real de validações com posicionamento SAE-IP.
- Processo de Monitorização de Alertas (UC09): análise contínua de eventos para deteção de anomalias com notificação em <30s.
- Processo de Consolidação Diária (UC10): agendado às 23h30, calculando KPIs e persistindo no Data Lake.
- Cálculo Matriz O-D (UC08): execução diária, disponibilizando matriz antes das 07h00.
- Servidor HTTP Spring Boot: processo contínuo servindo pedidos REST do Frontend e sistemas externos.

Mecanismos de Alta Disponibilidade e Degradação Graciosa

- Indisponibilidade OSM: Leaflet serve tiles a partir da cache local do browser; marcadores e dados LIVE DATA mantêm-se acessíveis (UC06).

- Falha no log de auditoria: acesso ao sistema é bloqueado — sem log, sem acesso (UC01, fail secure).
- Falha de validação JWT: acesso negado sem exposição de informação de diagnóstico ao utilizador (UC01).
- Data Lake indisponível: dados válidos enfileirados localmente com até 5 tentativas automáticas de reenvio a cada 30 segundos (UC02).
- Falha de integração ERP: fila local com retry automático a cada 15 minutos até restabelecimento (UC11).
- SAE-IP indisponível: última posição conhecida utilizada com marcação ‘localização estimada’ (UC07).

9.3 Vista de Casos de Uso

A Vista de Casos de Uso apresenta os doze casos de uso do sistema TickeTUB e os respetivos atores e descrições.

ID	Caso de Uso — Descrição
UC01	Autenticar utilizador via SSO — Autenticação delegada ao Auth0 com OAuth2/OIDC, validação JWT, atribuição RBAC e registo de logs de auditoria. Princípio fail secure.
UC02	Ingerir dados de bilhética — Endpoint REST que autentica o SCB, aceita picagens via HTTP POST com resposta HTTP 202, e processa de forma assíncrona: validação, anonimização RGPD, normalização NGSI-LD e persistência no Data Lake com garantia de entrega.
UC03	Categorizar por perfil de utilizador — Classificação automática por tipologia de título com tabela de mapeamento editável e governança RGPD pelo DPO.
UC04	Navegar na dashboard da aplicação — Dashboard operacional com widgets de procura em tempo real, estado de ingestão, alertas ativos e atalhos para módulos. Conteúdo adaptado ao perfil RBAC.

UC05	Consultar procura em tempo real — Análise integrada por horário, rota/linha e zona/paragem com filtros combinados e atualização contínua.
UC06	Consultar mapa de rede — Mapa interativo OSM/Leaflet com marcadores georreferenciados das paragens TUB e popups LIVE DATA com dados de bilhética em tempo real.
UC07	Correlacionar com dados operacionais — Cruzamento em tempo real de validações com GPS/SAE-IP e análise de desvio entre serviço planeado (GTFS) e executado.
UC08	Estimar origem-destino — Cálculo diário da matriz O-D com base em validações georreferenciadas, disponível antes das 07h00, com visualização no mapa e exportação CSV.
UC09	Gerar alertas e exceções — Detecção automática de anomalias (expirado, duplicado, evasão) com notificação em <30s, gestão por equipa de fiscalização e mapa de calor semanal.
UC10	Consultar histórico de utilização — Agregação histórica incremental em tempo real, séries temporais com comparação de períodos e exportação de relatórios para direção.
UC11	Assegurar integração com o planeamento — Simulação de cenários de ajuste de rede e integração automática com ERP para controlo financeiro de bilhética.
UC12	Exportar dados e garantir interoperabilidade — API NGSI-LD documentada em OpenAPI 3.0 com exportação em CSV/JSON restrita a dados aprovados pelo DPO.